

# IBC v2: Enabling IBC Everywhere



Susannah Evans

February 20, 2025

🕒 11 Min

Announcements

Code Walkthrough

Table Of Contents



The v2 of the Inter-Blockchain Communication Protocol, launches at the end of March 2025 with the most impactful improvements to the protocol since its initial launch in April 2021. This blog details why these changes were made, what they are, and how you can benefit from the newly improved design.

TL;DR

- **IBC v2 is the evolution of IBC**, significantly simplifying the protocol while maintaining expressiveness. V2 makes it easier to connect with diverse blockchain ecosystems, starting with Ethereum and Solana to



- **Streamlined design removes the complex handshakes of IBC Classic**, reducing implementation overhead and enabling faster, more flexible cross-chain connectivity, even for ecosystems without light clients.

- **ZK-enabled verification lowers costs**, making IBC connections to Ethereum affordable while maintaining security, with IBC Ethereum connections launching end of March 2025.

---

## Why do we need to redesign the IBC Protocol?

IBC, a secure, permissionless interoperability protocol that enables trust-minimized cross-chain communication with no middleman, launched ~ 3 years ago and has seen great success within the Cosmos ecosystem. It facilitates transfer volumes of up to **\$3Bn / month** over 115+ chains.

However, the expansion of IBC has been limited to relatively homogeneous blockchains, with the majority using CometBFT and the Cosmos SDK. Yet the blockchain space is vast with a variety of VMs, consensus algorithms, and applications frameworks utilized across blockchains such as Solana, Bitcoin, or Ethereum.

The burden of implementing IBC, with its complex, multiple layers of abstraction and verbose handshake protocols, has stifled its extensibility to new ecosystems. It was unfeasible to implement a light client, connection handshake, and channel handshake within a strictly gas-metered domain, such as the EVM, and would require years of development work to bring a new connection to market. Ultimately, IBC Classic was not fit for purpose to fulfill the dream of being a universal interoperability standard.

How do we enable IBC to extend beyond its current limitations? By keeping the best parts of the existing protocol and doing away with complexity. IBC v2 will enable IBC to fulfill its goal of becoming an interoperability standard that extends far beyond Cosmos, including a solution for chains that do not have a light client.

---

## What is IBC v2?

IBC v2 is the iteration of the core protocol that makes it significantly easier to implement across a wide variety of VMs and blockchain architectures. It retains many of the elements users love:

- Light client-based security model, with the ability to add other, flexible client types with different security models, i.e. a multi-sig based client.
- Permissionless access to the network.
- Expressive application semantics: Send, Receive, Acknowledge and Timeout Packets.
- Battle-tested, highly secure, and rigorously designed protocol.

- More diverse blockchains can use IBC as the requirements on a blockchain for a functional implementation have been reduced - namely removal of the need for a chain to introspect its own consensus state used during the IBC classic connection handshake, and removal of the requirement to use IBC protocol encoding.
- Enhanced upgradability as all applications and application versions are handled over a single connection, removing the need for channel upgradability.

The initial launch of IBC v2 will offer light client-backed connectivity for token transfers between Ethereum and Cosmos, with Solana to follow soon.

## How IBC v2 works

IBC v2 simplifies IBC into three core components: clients, a router, and applications:

### Clients

The client tracks the consensus state of a chain you want to connect with, and a relay uses it to verify that a certain action took place on the blockchain you want to connect with. IBC has been used extensively with light clients: on-chain, lightweight representations of blockchains. However, the IBC client layer is flexible enough to support verification of a single signature, a multi-sig, or clients that depend on another, i.e.

conditional clients.

### Router

The router is responsible for connecting two clients, registering each light client to a corresponding counterparty, and registering applications to a specific port ID. It also handles packet sequence replay protection and timeout checks and passes the proof, proof height, and expected path to the client for verification. After the verification logic is complete, packets will be routed to the relevant application.

The functionality of the router condenses the abstractions of IBC classic channels, connections, and the client router into a single abstraction layer, which does away with the complicated handshakes of IBC Classic. This enhances the upgradability of the protocol, as users can easily deploy and route to new applications, rather than coordinating a new channel handshake between chains. Application developers also benefit from simpler development as support for the channel handshake does not need to be implemented by a smart contract, only for the packet lifecycle. For example, in CosmWasm, this removes the need to support the `ibc_channel_open` and `ibc_channel_connect` entrypoints for a cross-chain contract.

### Applications

The application is responsible for all application specific logic, for token transfer this includes escrowing and minting tokens. At launch, token transfers and the middleware design pattern will be supported. After launch, cross-chain contract call applications will be added.





IBC Classic versus IBC v2.

## Enhanced application upgradability and flexibility through the introduction of the Payload concept

The format of IBC packets is improved, as the redundancy in the PortID, ChannelID tuple is removed, timeout height as a concept is removed and only timestamps remain. The application-specific packet data is replaced with a payload, which contains the version of an application and the encoding type. This means users can choose different encoding types: for example between Cosmos and Ethereum ABI encoding is used for reduced gas costs. The packet struct contains a list of payloads to enable a single packet to leverage functionality from different applications, which will be possible in future releases. This code snippet shows IBC v2 Packet and Payload structs from the solidity codebase.

```
1  /// @notice Packet struct
2  /// @param sequence The sequence number of the packet
3  /// @param sourceClient The source client identifier (client id)
4  /// @param destClient The destination client identifier
5  /// @param timeoutTimestamp The timeout timestamp in the counterparty chain, in unix seconds
6  /// @param payloads The packet payloads
7  struct Packet {
8      uint32 sequence;
9      string sourceClient;
10     string destClient;
```





The Inter-Blockchain  
Communication  
Protocol



```
11      uint64 timeoutTimestamp;
12      Payload[] payloads;
13  }
14
15      /// @notice Payload struct
16      /// @notice Used in the Packet struct and handled by IBC applications
17      /// @param sourcePort The source port identifier
18      /// @param destPort The destination port identifier
19      /// @param version The application version of the packet data
20      /// @param encoding The encoding of the packet data (value)
21      /// @param value The packet data
22      struct Payload {
23          string sourcePort;
24          string destPort;
25          string version;
26          string encoding;
27          bytes value;
28      }
```

**Simplified connection creation and startup with usability benefits**



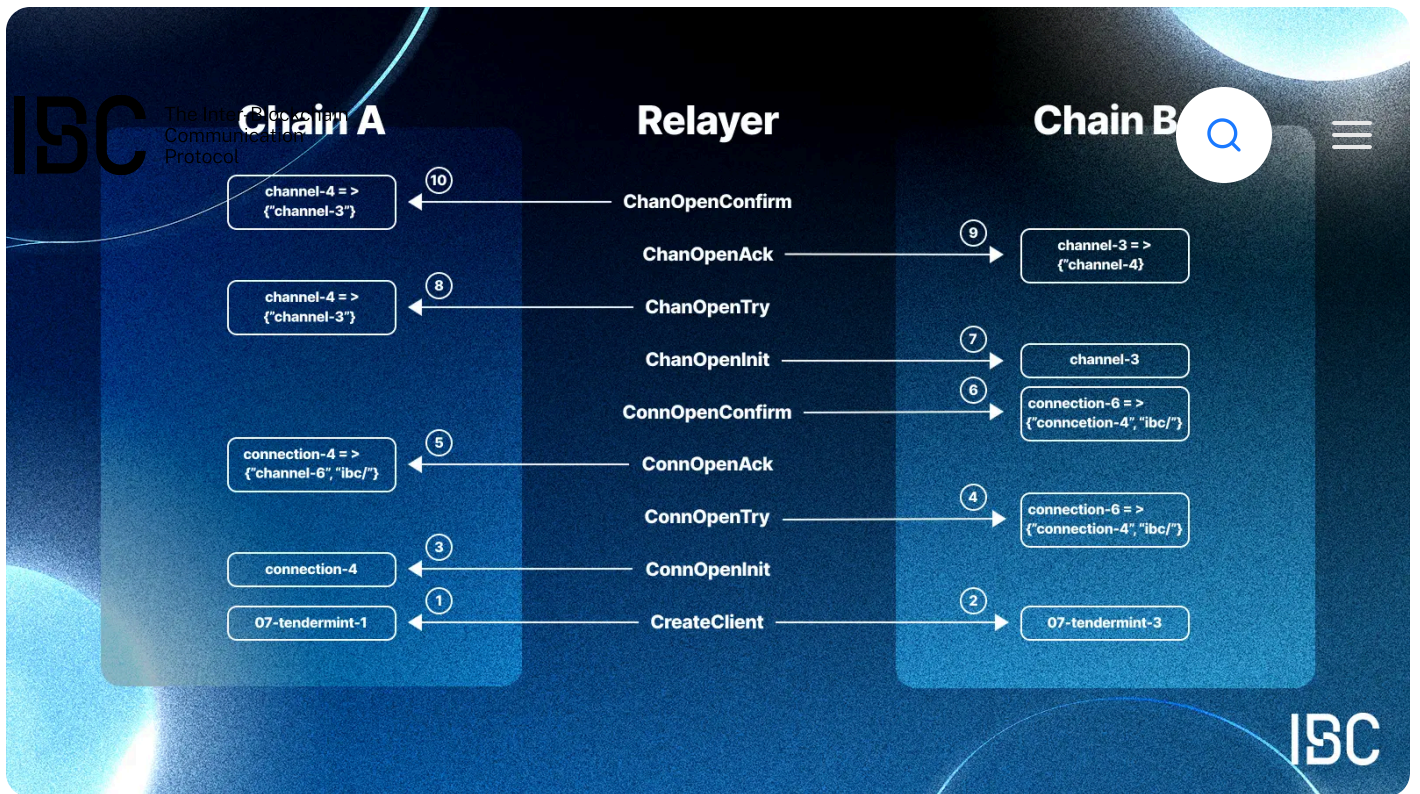
*Pictured above: IBC v2 Startup: Create a client, create and register the counterparty in a single step, then register the counterparty client on the original chain.*

IBC v2 massively simplifies connection startup; **instead of a 10-step startup process** consisting of a 4-step connection handshake, a 4-step channel handshake, and a step on each chain to create a light client, this process is simplified to creating a client on one chain, creating and registering a client on the counterparty (in a single step), and finally registering the counterparty client. This reduces the startup to 3 steps and reduces the provable surface area of the protocol: each step in the connection and channel handshake requires a relay to submit a proof alongside a message to progress the handshake. This makes implementing IBC v2 significantly easier.

The security model of IBC v2 is not compromised by reducing the provable surface area because social consensus is still used to determine which chain is associated with a certain channel, considered the canonical route. Because multiple channels can be created upon the same connection, this is particularly important for token transfer, as tokens are only fungible when they have travelled the same route -across the same channel. This path dependence avoids a malicious chain being able to infinitely mint tokens imitating a legitimate token. However, it creates usability issues: between the Cosmos Hub and Osmosis, there are **40 channels**, but channel-141 is considered to be the canonical channel, driven by social consensus.

Provided that the complex 10 step startup process in IBC classic, ultimately relies on social consensus, IBC v2 does away with this complexity. Social consensus is used to determine which client is considered to be a certain blockchain. Packets flow across a pair of clients, this is similar to imagining a single channel between connected chains enhancing the usability of token transfer.



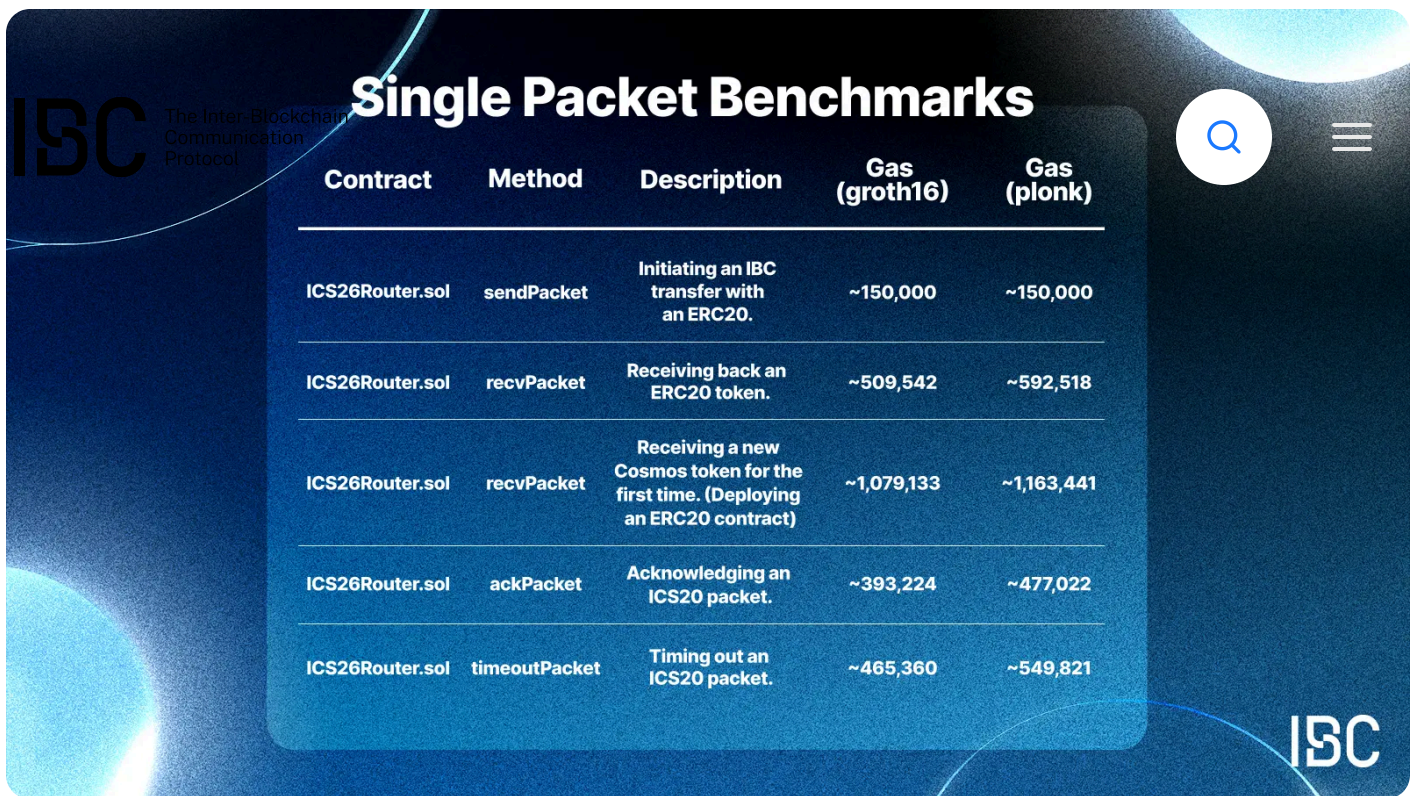


Pictured above: IBC Classic start up: Client creation, connection handshake and channel handshake.

## Leveraging ZK to reduce Tendermint light client verification costs

IBC v2 provides affordable IBC transactions to Ethereum. It accomplishes this through the use of **Succinct's Processor SP1**, a zkVM that runs a Rust-based tendermint light client to verify the consensus state of a Cosmos chain, and inclusion or exclusion of IBC packets. An illustrative cost of sending tokens from Ethereum to a Cosmos chain, using a gas cost of 2.5 gwei, ETH cost of \$2600, and 149,000 gas is only \$0.97. Note that only receive's and acknowledgments require verification using a ZK proof, as highlighted in the benchmarks table.





*Pictured above: Benchmarks for the v2 solidity implementation sending and receiving IBC packets as standalone packets or batched. Benchmarks as of February 20, 2025.*



---

# When will IBC v2 be ready?

Interchain Labs  
The Inter-Blockchain  
Communication  
Protocol



An audit is underway in preparation for the protocol to launch at the end of March 2025. Over time, the entire IBC network will migrate to IBC v2, gradually rolled out when both sides of a connection are leveraging the protocol. Consequently, **ibc-go v10**, the launch version for Cosmos chains compatible with SDK 0.50 and Cometbft 0.38 lines, still supports IBC classic and IBC v2 connections. The IBC **solidity implementation** will be released and deployed on Ethereum alongside the Cosmos release.

Want Day 1 access to v2? Reach out to the **IBC Product Manager Susannah Evans** to get more information on the protocol's technical implementation and deployment considerations.

---

## About Interchain Labs

**Interchain Labs** (formerly Skip Protocol) is building the development platform for world-scale multichain application builders: people with a giant vision who need a tech stack that gives them that expressivity and scale. Labs is building the tools essential for the next generation –enabling new, fairer ways of organizing our communities, our lives, and the world.



**Susannah Evans**

IBC Product Lead.



## Recent Articles





[Announcements](#)[Code Walkthrough](#)[About IE](#)

## IBC v2: Enabling IBC Everywhere

The v2 of the Inter-Blockchain Communication Protocol, launches at the end of March 2025. This blog details the changes to the protocol and how you can benefit from the improved design.



**Susannah Evans**

February 19, 2025

🕒 11 Min



## IBC A

This po  
working  
are see

Head to our Github to begin.

**Ready to get started?**

[Start Building](#)[Read the Docs](#)

## The Inter-Blockchain Communication Protocol

Ready to get started? Head to our [Github](#) to begin.

[Start Building](#)

# Resources

**ISBC** The Inter-Blockchain  
Communication  
Protocol  
Specification



Reference Implementation

Developer Portal

Learning Videos

Skip Go

Insights

Join the Community



The Inter-Blockchain Communication Protocol is supported by the Interchain Foundation.

Privacy Policy